

Problem 1

Let $G = (V, E)$ be a directed acyclic graph where each vertex $v \in V$ is associated with a number of stars between 1 and 5, denoted by $\star(v) \in \{1, 2, \dots, 5\}$, and let $s, t \in V$ be distinct vertices.¹ Describe and analyze an $O(V + E)$ -time algorithm to compute the shortest path from s to t in G with at least total 330 stars among the vertices in the path, or report that there is no such path. [Hint: Compute a topological ordering and use it to define a valid evaluation order.]

Solution: Let $SSP(u, x)$ be the length of the shortest path from u to t with at least x stars.

$$SSP(u, x) = \begin{cases} 0 & u = t \text{ and } x \leq \star(t) \\ \infty & u = t \text{ and } x > \star(t) \\ \min_{u \rightarrow v \in E} 1 + SSP(v, \max\{0, x - \star(v)\}) & \text{otherwise} \end{cases}$$

The min function over the empty set returns ∞ here, i.e., when u has no out-neighbors in the recursive case.

We first index the vertices using their position in topological order, from 1 to n . We memoize using a 1D array $M[1..n][0..330]$, where $M[u][x]$ stores $SSP(u, x)$ for all $u \in V$ and $0 \leq x \leq 330$. Our evaluation is described with the following nested-for loops:

```
for u from n downto 1:
  for x from 0 to 330:
```

There are 331 subproblems for each vertex $u \in V$, which takes $O(\text{outdegree}(u))$ time each. Hence the overall runtime is $O(V + E)$ to compute the topological ordering via DFS, followed by

$$\sum_{u \in V} O(\text{outdegree}(u)) = O(E)$$

time, which is $O(V + E)$ overall.

To obtain the number of edges in the shortest desired path, output $SSP(s, 330)$. To retrieve the edges that compose the desired path, we use standard backtracking techniques as we saw for the edit distance and line-breaking problems. In particular, for this problem, we maintain another array $succ[1..n][0..330]$ such that $succ[u][x]$ stores the “successor” pair of parameters (v, y) for which $SSP(u, x) = 1 + SSP(v, y)$ in the recursive case and a sentinel value, e.g., *null*, in the base cases. The sequence of successors starting with pair $(s, 330)$ yields the sequence of vertices that compose the path with length $SSP(s, 330)$, namely the first value in each successor pair. This process takes $O(V + E)$ additional time.

¹In $\text{\LaTeX}$, the star symbol $\star$ is $\text{\texttt{\$}\bigstar\text{\$}}$.

Problem 2

Let $G = (V, E)$ be a directed graph where each edge is *colored* grey or Duke blue, denoted by $c(e) \in \{g, b\}$, and let s, t be distinct vertices. Describe a $O(V + E)$ -time algorithm to determine if there is a walk in G from s to t where no three consecutive edges have the same color. For example, if the walk has two grey edges in a row, then either the walk must end or the next edge after those two grey edges must be blue. Justify its correctness and analyze its runtime complexity.

Solution: We solve this problem by reduction to graph reachability. We construct a directed graph $G' = (V', E')$ as follows:

$$\begin{aligned} V' &= (V \times \{\emptyset, g, b\} \times \{\emptyset, g, b\}) \cup \{T\} \\ E' &= \{(u, x, y) \rightarrow (v, y, c(e)) \mid e = u \rightarrow v \in E, x, y \in \{\emptyset, g, b\}, x \neq c(e) \text{ or } y \neq c(e)\} \\ &\quad \cup \{(u, x, y) \rightarrow T \mid u \in V\} \end{aligned}$$

That is, for each vertex, we create six copies, where the second value represents the color of the second-to-last edge taken, and the third value represents the last color of the last edge taken; if no such edge taken, this is represented with “color” $\emptyset$ for that edge.

The key property is that, by construction of G' , there is a valid walk in G from s to t if and only if there is a path in G' from $(s, \emptyset, \emptyset)$ to T .

Thus we reduce the problem to determining whether there is a path from $(s, \emptyset, \emptyset)$ to T in G' , which is solved using DFS in $O(V' + E')$ time. We report True if indeed T is reachable from $(s, \emptyset, \emptyset)$ and report False otherwise.

G' has $6V + 1$ vertices and at most $6E + 6 = O(E)$ edges, and G' can be constructed in time proportional to its number of vertices and edges. Thus the total runtime of the algorithm is $O(V' + E') = O(V + E)$.

Problem 3

Alex is shopping for a new house in another city. Its road network is represented by an undirected graph $G = (V, E)$ with positive edge weights given by $w : E \rightarrow \mathbb{R}^+$, where the vertices are points-of-interest, and edges are direct road segments between them, and $w(e)$ for an edge e is the amount of time in minutes it takes to travel down (following the speed limit). (There are no one-way streets, so modeling the edges as undirected suffices.)

Let $H \subset V$ be a subset of houses for sale, let $D \subset V$ be the subset of daycares, and let $p \in V$ be the parking lot for his new job's building. Every weekday, Alex has to drop his son off at a daycare on his way to work, then pick his son up (at the same daycare) on his way home after work. Also interested in minimizing his commute, Alex wants to find the house $h \in H$ with the shortest total driving time each day. That is, the shortest drive from a house $h \in H$ to a daycare $d \in D$, then to p , then back to d , and finally back to h .

Describe an $O(E + V \log V)$ -time algorithm to find the desired house for Alex. Justify its correctness and analyze its runtime complexity.

[Hint: First, would Alex use different routes to and from work? Second, ideas from the graph reductions in Discussion 5 may be useful here.]

Solution: (A reduction with a single call to Dijkstra's, in a different graph) We solve this problem by reduction to shortest paths. The total travel time consists of four parts: the time from a house to a daycare, from that daycare to the parking lot p , and then *the same walk in reverse*. As we will see, we actually find the shortest walk, starting at p , back to a house $h \in H$; its reverse is the first part of the shortest desired route.

We construct an undirected graph $G' = (V', E')$ as follows:

$$\begin{aligned} V' &= \{v_0, v_1 \mid v \in V\} \\ E' &= \{u_i v_i \mid uv \in E, i \in \{0, 1\}\} \\ &\quad \cup \{d_0, v_1 \mid d \in D, dv \in E\} \end{aligned}$$

That is, we add a single super-source vertex S that is connected to each house. The weights of existing edges are the same, and the new edges have weight zero.

The key property is that, by construction of G' , the shortest walk from p to a house $h \in H$ that visits a daycare $d \in D$ on its way has length k if and only if the shortest path from p_0 to h_1 has length k .

Thus, using Dijkstra's algorithm starting at p_0 in G' , we obtain for each house $h \in H$ the shortest length of a walk visiting a daycare $d \in D$, as desired. The total runtime is the time to construct G' , which takes $O(1)$ time per vertex and edge in G , followed by $O(E' + V' \log V') = O(E + V \log V)$ time using Fibonacci heaps. Thus the overall runtime is $O(E + V \log V)$.

Solution: (A reduction with multiple calls to Dijkstra's) We solve this problem by reduction to shortest paths. The total travel time consists of four parts: the time from a house to a daycare, from that daycare to the parking lot p , and then *the same walk in reverse*. As we will see, we actually find the shortest walk, starting at p , back to a house $h \in H$; its reverse is the first part of the shortest desired route.

Using Dijkstra's algorithm in G from p , we obtain the shortest path distances $\text{dist}(p, d)$ from p to each node in G ; in particular, to every daycare $d \in D$. What remains to find which daycare should be visited from p to then reach a house $h \in H$ with minimum total distance (i.e., travel time), **which includes the distance from p to that daycare**.

We construct an undirected graph $G' = (V', E')$ as follows:

$$\begin{aligned} V' &= V \cup \{S\} \\ E' &= E \cup \{Sd \mid d \in D\} \end{aligned}$$

That is, we add a single super-source vertex S that is connected to each daycare. The weights of existing edges are the same, and the new edges between S and each daycare $d \in D$ has weight $\text{dist}(p, d)$.²

The key property is that, by construction of G' , the shortest route among all houses $h \in H$ and $d \in D$ has length k if and only if the shortest path from S to any house (through a daycare) has length k .

Thus, using Dijkstra's algorithm starting at S in G' , we obtain the shortest path distances from S to each house $h \in H$ in G' . Let $h^* \in H$ be the house with the minimum distance, and let $d^* \in D$ be the daycare incident to S on the corresponding shortest path to h^* . The desired path in G is then the shortest path in G from h^* to d^* , followed by the shortest path from d^* to p , followed by this overall walk from h^* to p in reverse.

The total runtime is the time to run Dijkstra's once in G , the time to construct G' , and then the time to run Dijkstra's once in G' . Since G' has complexity $O(V + E)$, the overall runtime is $(E + V \log V + E' + V' \log V') = O(E + V \log V)$ time using Dijkstra's implemented with Fibonacci heaps.

Problem 4

There are n airports, each given an index from 1 to n . There are m flights between pairs of airports each day, where at most F flights depart from each airline. Each j -th flight is represented by a 4-tuple (d_j, a_j, t_j, ℓ_j) , where:

- $d_j \in \{1, \dots, n\}$ is the airport from the flight departs,
- $a_j \in \{1, \dots, n\}$ is the airport at which the flight arrives,
- $t_j \in \{0, 1, \dots, 1439\}$ is the number of minutes after midnight that the flight departs from airport d_j (e.g., $t_j = 1013$ corresponds to 4:53 PM and $t_j = 1439$ corresponds to 11:59 PM), and
- $\ell_j \in \{0, 1, \dots, 1439\}$ is the number of minutes after midnight that the flight arrives at airport a on the same day.

Note that, by definition, $0 \leq d_j < \ell_j < 1440$, and no flight is midair at midnight; all flights land on the same day that they depart. For example, $(9, 3, 653, 1136)$ represents a flight from airport 9 to airport 3 that departs at 10:53 AM and lands at 6:56 PM.

²A common error, even by modern LLMs like Claude and ChatGPT, is to use zero-weight edges here. However, it is impossible to obtain the overall solution from only the shortest-path distances from p in G and from S in G' with the zero-weight edges: the optimal house and daycare may not be a closest house-daycare pair and these shortest-path distances give no information about such pairs.

Suppose you wish to find the earliest time that you can arrive at airport n , assuming you begin at airport 1 at midnight (time 0), by taking a sequence of flights (or possibly a direct one). As in real life, you can only take a flight that departs from an airport i if you are presently at that airport before the flight departs. Describe an algorithm to find this earliest arrival time, whose input is the number of airports, n , and the m tuples representing flights, where at most F flights depart from each airport. Your algorithm should be as fast as you can achieve in terms of n, m , and F .³ Justify its correctness and analyze its runtime complexity.

Solution: (Without exploiting constant-size universe of departure/arrival times.) We reduce this problem to shortest paths (and then reachability to improve the runtime). Let Q be the list of 4-tuples representing the flights. For each airport i , let T_i be the set of all flight departure times from airport i , and let L_i be the flight arrival times at airport i . In particular,

$$T_i = \{t_j \mid (1, a_j, t_j, \ell_j) \in Q\} \quad \text{and} \quad L_i = \{\ell_j \mid (d_j, 1, t_j, \ell_j) \in Q\}.$$

Note that, for each flight tuple $(d_j, a_j, t_j, \ell_j) \in Q$, the times t_j, ℓ_j appear twice among the T_i 's and L_i 's. That is, $\sum_i |T_i| + |L_i| \leq 2m$.

For any time x , let $\text{next}_i(x)$ be the time of the earliest flight departure after t ; if there is no such flight, it is ∞ . By sorting each of the sets T_i in increasing order, $\text{next}_i(x)$ can be computed in $O(\log |T_i|) = O(\log F)$ time.

We construct a graph $G = (V, E)$ where:

$$\begin{aligned} V &= \{1, 0\} \cup \{(i, x_j) \mid i \in \{1, \dots, n\}, x_j \in T_i \cup L_i\} \\ E &= \{(d_j, t_j) \rightarrow (a_j, \ell_j) \mid (d_j, t_j, a_j, \ell_j) \in Q\} \\ &\quad \cup \{(i, x_j) \rightarrow (i, \text{next}_i(x_j)) \mid i \in \{1, \dots, n\}, x_j \in T_i \cup L_i, \text{next}_i(x_j) \neq \infty\} \end{aligned}$$

That is, for every airport i , we create a pair of i with each arrival time of flights to i and each departing time of flights from i . The first set of edges in E correspond to flights from one airport to another, and the second set of edges in E correspond to time elapsing from times in $T_i \cup L_i$ to the next earliest time in L_i for each airport i .

The key property of G is that the minimum travel time from airport 1 at time 0 is equal to x , the smallest time of the reachable vertices (if any) of the form (n, x) from $(0, 0)$ in G .

(We note that we could also weight each edge $(a, b) \rightarrow (c, d) \in E$ to be $d - b$, the amount of time elapsed from time b to time d , but running Dijkstra's algorithm in G to find the shortest path to any node of the form (n, x) would take (slightly) longer than our DFS-based algorithm below, and in fact, for this graph the sum of weights would be equal to x for such a node.)

Thus the problem is solved using DFS (or BFS) from $(1, 0)$ to identify which nodes with airport n can be reached. This takes $O(V + E)$ time. G has $1 + \sum_i |T_i| + |L_i| = 2m + 1$ vertices. For each airport i , a node of the form (i, x_j) for $x_j \in T_i \cup L_i$ has out-degree at most 1 if $x_j \in L_i$ and at most 2 if $x_j \in T_i$. Thus, the total number of edges is equal to the sum of out-degrees of all nodes, which is at most $O(V) = O(m)$. To construct the edges of G , it takes $O(1)$ time per edge in the first set, and $O(\log F)$ time per edge in the second set. Indeed, $|T_i| \leq F$ for each airport i , and after sorting all T_i 's

³Because this is a graph reduction problem, the runtime will depend on the size of the graph you construct. First obtain a correct reduction, then consider how to further improve the construction/runtime after.

in $\sum_i |T_i| \log |T_i| = O(m \log F)$ time, we can compute $\text{next}_i(x)$ for any time x in $O(\log F)$ time. We conclude that the overall runtime is dominated by the time to construct G , $O(m \log F)$ time.

Solution: (Using DFS by exploiting the constant-size universe of departure/arrival times.) We reduce this problem to graph reachability. Let Q be the list of 4-tuples representing the flights. We construct a graph $G = (V, E)$ where:

$$\begin{aligned} V &= \{(i, j) \mid i \in \{1, \dots, n\}, j \in \{0, \dots, 1439\}\} \\ E &= \{(d_j, t_j) \rightarrow (a_j, \ell_j) \mid (d_j, t_j, a_j, \ell_j) \in L\} \\ &\quad \cup \{(i, j) \rightarrow (i, j+1) \mid i \in \{1, \dots, n\}, j \in \{0, \dots, 1438\}\} \end{aligned}$$

That is, for every airport i , we create a vertex for each time in minutes from 0 to 1439. The first set of edges in E correspond to flights, and the second set of edges in E correspond to a single minute elapsing while at the same airport.

The key property of G is that the minimum travel time from airport 1 at time 0 is equal to x , where x is the smallest time of the reachable vertices (if any) of the form (n, x) from $(0, 0)$ in G .

Thus the problem is solved using DFS (or BFS) from $(1, 0)$ to identify which nodes with airport n can be reached. This takes $O(V' + E') = O(V + E)$ time, noting that $|V| = n$ and $|E| = |L| + 1440n = O(m + n)$. G is constructed in the same time, so the overall runtime is $O(n + m)$.